

**Федеральное государственное автономное образовательное
учреждение высшего образования
«Национальный исследовательский университет ИТМО»
Факультет Программной Инженерии и Компьютерной Техники**

Блок задач с Яндекс.Контеста №4
Алгоритмы и Структуры Данных

Выполнил:
Ларионов Владислав Васильевич
Группа Р3209

Проверил:
Косяков Михаил Сергеевич

Санкт-Петербург, 2026

Содержание

М. Цивилизация	3
Условие:	3
Решение:	4
О решении:	6
Н. Свинки-копилки	7
Условие:	7
Решение:	8
О решении:	9
О. Долой списывание!	10
Условие:	10
Решение:	11
О решении:	12
Р. Авиаперелеты	13
Условие:	13
Решение:	14
О решении:	16

М. Цивилизация

Условие:

Карта мира в компьютерной игре «Цивилизация» версии 1 представляет собой прямоугольник, разбитый на квадратики. Каждый квадратик может иметь один из нескольких возможных рельефов, для простоты ограничимся тремя видами рельефов — поле, лес и вода. Поселенец перемещается по карте, при этом на перемещение в клетку, занятую полем, необходима одна единица времени, на перемещение в лес — две единицы времени, а перемещаться в клетку с водой нельзя.

У вас есть один поселенец, вы определили место, где нужно построить город, чтобы как можно скорее завладеть всем миром. Найдите маршрут переселенца, приводящий его в место строительства города, требующий минимального времени. На каждом ходе переселенец может перемещаться в клетку, имеющую общую сторону с той клеткой, где он сейчас находится.

Формат ввода

Во входном файле записаны два натуральных числа N и M , не превосходящих 1000 — размеры карты мира (N — число строк в карте, M — число столбцов). Затем заданы координаты начального положения поселенца x и y , где x — номер строки, y — номер столбца на карте ($1 \leq x \leq N, 1 \leq y \leq M$), строки нумеруются сверху вниз, столбцы — слева направо. Затем аналогично задаются координаты клетки, куда необходимо привести поселенца.

Далее идет описание карты мира в виде N строк, каждая из которых содержит M символов. Каждый символ может быть либо «.» (точка), обозначающим поле, либо «W», обозначающим лес, либо «#», обозначающим воду. Гарантируется, что начальная и конечная клетки пути переселенца не являются водой.

Формат вывода

В первой строке выходного файла выведите количество единиц времени, необходимое для перемещения поселенца (перемещение в клетку с полем занимает 1 единицу времени, перемещение в клетку с лесом — 2 единицы времени). Во второй строке выходного файла выведите последовательность символов, задающих маршрут переселенца. Каждый символ должен быть одним из четырех следующих: «N» (движение вверх), «E» (движение вправо), «S» (движение вниз), «W» (движение влево). Если таких маршрутов несколько, выведите любой из них.

Если дойти из начальной клетки в конечную невозможно, выведите число — 1.

Пример

Ввод	Вывод
4 8 1 1 4 8 ...WWW .#####. .#..W... ...WWW.	13 SSSENEEEEEES

Решение:

```
#include <algorithm>
#include <iostream>
#include <queue>
#include <string>
#include <vector>

int main() {
    int rows, cols;
    int start_row, start_col;
    int end_row, end_col;
    std::cin >> rows >> cols >> start_row >> start_col >> end_row >> end_col;

    std::vector<std::string> map;
    map.reserve(rows);
    for (int i = 0; i < rows; i++) {
        std::string row;
        std::cin >> row;
        map.push_back(row);
    }

    start_row--;
    start_col--;
    end_row--;
    end_col--;

    std::vector<std::vector<int>> distances(rows, std::vector<int>(cols));
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            if (!(i == start_row && j == start_col)) {
                distances[i][j] = 1000000000;
            }
        }
    }

    std::vector<std::vector<std::pair<int, int>>> parents(
        rows, std::vector<std::pair<int, int>>(cols)
    );
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            parents[i][j] = {-1, -1};
        }
    }

    std::priority_queue<
        std::pair<int, std::pair<int, int>>,
        std::vector<std::pair<int, std::pair<int, int>>>,
        std::greater<std::pair<int, std::pair<int, int>>>>
        min_distance;
    min_distance.push({
        0, {start_row, start_col}
    });
```

```

std::vector<int> neighbour_rows = {1, -1, 0, 0};
std::vector<int> neighbour_cols = {0, 0, -1, 1};

while (min_distance.size() != 0) {
    std::pair<int, std::pair<int, int>> curr = min_distance.top();
    int curr_distance = curr.first;
    int curr_row = curr.second.first;
    int curr_col = curr.second.second;
    min_distance.pop();

    if (curr_distance != distances[curr_row][curr_col]) {
        continue;
    }

    for (int i = 0; i < 4; i++) {
        int neighbour_row = curr_row + neighbour_rows[i];
        int neighbour_col = curr_col + neighbour_cols[i];

        bool inside_map =
            0 <= neighbour_row && neighbour_row < rows && 0 <= neighbour_col &&
neighbour_col < cols;
        if (!inside_map) {
            continue;
        }

        if (map[neighbour_row][neighbour_col] == '#') {
            continue;
        }

        int new_distance;
        if (map[neighbour_row][neighbour_col] == '.') {
            new_distance = curr_distance + 1;
        } else {
            new_distance = curr_distance + 2;
        }

        if (new_distance < distances[neighbour_row][neighbour_col]) {
            distances[neighbour_row][neighbour_col] = new_distance;
            parents[neighbour_row][neighbour_col] = {curr_row, curr_col};

            min_distance.push({
                new_distance, {neighbour_row, neighbour_col}
            });
        }
    }
}

if (distances[end_row][end_col] == 1000000000) {
    std::cout << -1 << std::endl;
} else {
    std::cout << distances[end_row][end_col] << std::endl;
}

```

```

std::string path;
int curr_row = end_row;
int curr_col = end_col;
while (curr_row != start_row || curr_col != start_col) {
    int prev_row = parents[curr_row][curr_col].first;
    int prev_col = parents[curr_row][curr_col].second;

    if (prev_row == curr_row + 1) {
        path += 'N';
    } else if (prev_col == curr_col - 1) {
        path += 'E';
    } else if (prev_row == curr_row - 1) {
        path += 'S';
    } else if (prev_col == curr_col + 1) {
        path += 'W';
    }

    curr_row = prev_row;
    curr_col = prev_col;
}

std::reverse(path.begin(), path.end());
std::cout << path << std::endl;
}
}

```

О решении:

Идея:

Представляем карту как взвешенный граф, где поле или лес – вершина, воду не учитываем, а переход из вершины в вершину имеет вес соответственно условиям задачи. В качестве поиска наикратчайшего пути используется алгоритм Дейкстры из начальной точки, храним минимальные расстояния и предков для восстановления маршрута.

Сложность по времени:

$O(NM \log NM)$ – так как карта имеет размеры N на M , а Дейкстра на `priority_queue` работает за $O(V \log V)$ (у нас $V = NM$)

Сложность по памяти:

$O(NM)$ – так как храним карту размера N на M

Н. Свинки-копилки

Условие:

У Васи есть n свинок-копилков, свинки занумерованы числами от 1 до n . Каждая копилка может быть открыта единственным соответствующим ей ключом или разбита.

Вася положил ключи в некоторые из копилков (он помнит, какой ключ лежит в какой из копилков). Теперь Вася собрался купить машину, а для этого ему нужно достать деньги из всех копилков. При этом он хочет разбить как можно меньшее количество копилков (ведь ему еще нужно копить деньги на квартиру, дачу, вертолет...). Помогите Васе определить, какое минимальное количество копилков нужно разбить.

Формат ввода

В первой строке содержится число n — количество свинок-копилков ($1 \leq n \leq 100$). Далее идет n строк с описанием того, где лежит ключ от какой копилки: в i -й из этих строк записан номер копилки, в которой находится ключ от i -й копилки.

Формат вывода

Выведите единственное число: минимальное количество копилков, которые необходимо разбить.

Пример

Ввод 

4
2
1
2
4

Вывод 

2

Решение:

```
#include <iostream>
#include <unordered_set>
#include <vector>

int main() {
    int piggy_banks_count;
    std::cin >> piggy_banks_count;

    std::vector<int> keys(piggy_banks_count + 1);
    for (int i = 1; i < piggy_banks_count + 1; i++) {
        int key_from_piggy_i_in_here;
        std::cin >> key_from_piggy_i_in_here;
        keys[i] = key_from_piggy_i_in_here;
    }

    int cycles = 0;
    std::unordered_set<int> all_discovered_piggy_banks;

    for (int i = 1; i < piggy_banks_count + 1; i++) {
        int curr = i;
        std::unordered_set<int> currently_visited_piggy_banks;

        if (all_discovered_piggy_banks.contains(i)) {
            continue;
        }

        while (!all_discovered_piggy_banks.contains(curr)) {
            all_discovered_piggy_banks.insert(curr);
            currently_visited_piggy_banks.insert(curr);
            curr = keys[curr];
        }

        if (currently_visited_piggy_banks.contains(curr)) {
            cycles++;
        }
    }

    std::cout << cycles << std::endl;
}
```

О решении:

Идея:

Сроится функциональный граф – из каждой копилки идет ровно одно ребро в копилку, где лежит ее ключ. Минимальное число разбитых копилок = число циклов в графе, потому что необходимо разбить хотя бы одну копилку в цикле, чтобы получить первый ключ и открывать дальше. Если по окончании обхода копилок (через ключи в них, то есть открывая последовательно) приходим в вершину, откуда начинали, то $+1$ цикл, если уперлись в цикл, который уже был (проверка на пройденные вершины), то переходим к следующей копилке, что гарантирует нам факт открытия каждой копилки ровно один раз.

Сложность по времени:

$O(n)$ – так как каждая копилка посещается не более одного раза, а операции в `unordered_set` работают в среднем за $O(1)$

Сложность по памяти:

$O(n)$ – так как храним все входные данные, то есть массив ключей в копилках

О. Долой списывание!

Условие:

Во время теста Михаил Дмитриевич заметил, что некоторые лкшата обмениваются записками. Сначала он хотел поставить им всем двойки, но в тот день Михаил Дмитриевич был добрым, а потому решил разделить лкшат на две группы: списывающих и дающих списывать, и поставить двойки только первым.

У Михаила Дмитриевича записаны все пары лкшат, обменявшихся записками. Требуется определить, сможет ли он разделить лкшат на две группы так, чтобы любой обмен записками осуществлялся от лкшонка одной группы лкшонку другой группы.

Формат ввода

В первой строке находятся два числа N и M — количество лкшат и количество пар лкшат, обменивающихся записками ($1 \leq N \leq 100, 0 \leq M \leq \frac{N(N-1)}{2}$). Далее в M строках расположены описания пар лкшат: два различных числа, соответствующие номерам лкшат, обменивающихся записками (нумерация лкшат идёт с 1). Каждая пара лкшат перечислена не более одного раза.

Формат вывода

Необходимо вывести ответ на задачу Павла Олеговича. Если возможно разделить лкшат на две группы, выведите «YES»; иначе выведите «NO».

Пример

Ввод 

```
3 2
1 2
2 3
```

Вывод 

```
YES
```

Решение:

```
#include <iostream>
#include <queue>
#include <vector>

int main() {
    int students_count, pairs_count;
    std::cin >> students_count >> pairs_count;

    std::vector<std::vector<int>> students(students_count);
    for (int i = 0; i < pairs_count; i++) {
        int dummy_1, dummy_2;
        std::cin >> dummy_1 >> dummy_2;

        dummy_1--;
        dummy_2--;

        students[dummy_1].push_back(dummy_2);
        students[dummy_2].push_back(dummy_1);
    }

    bool can_detect_dummies = true;
    std::queue<int> students_to_color;
    std::vector<int> students_colored(students_count, 67);

    for (int i = 0; i < students_count; i++) {
        if (students_colored[i] != 67) {
            continue;
        } else {
            students_colored[i] = 0;
            students_to_color.push(i);

            while (students_to_color.size() != 0) {
                int curr_student = students_to_color.front();
                students_to_color.pop();

                for (int near_student : students[curr_student]) {
                    if (students_colored[near_student] == 67) {
                        students_colored[near_student] = 1 - students_colored[curr_student];
                        students_to_color.push(near_student);
                    } else {
                        if (students_colored[near_student] == students_colored[curr_student]) {
                            can_detect_dummies = false;
                            break;
                        }
                    }
                }
            }
        }
    }

    if (can_detect_dummies) {
```

```

    std::cout << "YES" << std::endl;
} else {
    std::cout << "NO" << std::endl;
}
}

```

О решении:

Идея:

Строим неориентированный граф по парам лкшат и проверяем, можно ли покрасить его в 2 цвета так, чтобы концы каждого ребра имели разные цвета. Если покрасить получилось, тогда считается, что разделить лкшат на 2 группы можно, если покрасить нельзя, то и разделить тоже. То есть, задача сводится к проверке графа на двудольность через обход в ширину.

Сложность по времени:

$O(N + M)$ – так как у нас у нас N вершин графа (лкшат) и M ребер (пар), каждая вершина помещается в очередь не более раза, каждое ребро просматривается константное число раз

Сложность по памяти:

$O(N + M)$ – так как у нас хранится список смежности + список покрашенных лкшат размера N

Р. Авиаперелеты

Условие:

Главного конструктора Петю попросили разработать новую модель самолёта для компании «Air Бубундия». Оказалось, что самая сложная часть заключается в подборе оптимального размера топливного бака.

Главный картограф «Air Бубундия» Вася составил подробную карту Бубундии. На этой карте он отметил расход топлива для перелёта между каждой парой городов.

Петя хочет сделать размер бака минимально возможным, для которого самолёт сможет долететь от любого города в любой другой (возможно, с дозаправками в пути).

Формат ввода

Первая строка входного файла содержит натуральное число n ($1 \leq n \leq 1\,000$) — число городов в Бубундии. Далее идут n строк по n чисел каждая. j -е число в i -й строке равно расходу топлива при перелёте из i -го города в j -й. Все числа не меньше нуля и меньше 10^9 . Гарантируется, что для любого i в i -й строчке i -е число равно нулю.

Формат вывода

Первая строка выходного файла должна содержать одно число — оптимальный размер бака.

Пример

Ввод 

```
4
0 10 12 16
11 0 8 9
10 13 0 22
13 10 17 0
```

Вывод 

```
10
```

Решение:

```
#include <algorithm>
#include <iostream>
#include <queue>
#include <unordered_set>
#include <vector>

int find_max(std::vector<std::vector<int>>& map) {
    int max_fuel = 0;
    for (const auto& row : map) {
        auto it_max = std::max_element(row.begin(), row.end());
        max_fuel = std::max(*it_max, max_fuel);
    }

    return max_fuel;
}

bool can_fly_with_X_fuel(std::vector<std::vector<int>>& map, int X_fuel, int
cities_count) {
    std::unordered_set<int> were_in_cities;
    std::queue<int> cities_to_discover;

    were_in_cities.insert(0);
    cities_to_discover.push(0);
    while (cities_to_discover.size() != 0) {
        int curr_city = cities_to_discover.front();
        cities_to_discover.pop();

        for (int next_city = 0; next_city < cities_count; next_city++) {
            if (map[curr_city][next_city] <= X_fuel) {
                if (!were_in_cities.contains(next_city)) {
                    were_in_cities.insert(next_city);
                    cities_to_discover.push(next_city);
                }
            }
        }
    }

    if (static_cast<int>(were_in_cities.size()) != cities_count) {
        return false;
    }

    were_in_cities.clear();

    were_in_cities.insert(0);
    cities_to_discover.push(0);
    while (cities_to_discover.size() != 0) {
        int curr_city = cities_to_discover.front();
        cities_to_discover.pop();

        for (int next_city = 0; next_city < cities_count; next_city++) {
            if (map[next_city][curr_city] <= X_fuel) {
```

```

        if (!were_in_cities.contains(next_city)) {
            were_in_cities.insert(next_city);
            cities_to_discover.push(next_city);
        }
    }
}

if (static_cast<int>(were_in_cities.size()) != cities_count) {
    return false;
}

return true;
}

int main() {
    int cities_count;
    std::cin >> cities_count;

    std::vector<std::vector<int>> fuel_between_cities(cities_count,
std::vector<int>(cities_count));
    for (int i = 0; i < cities_count; i++) {
        for (int j = 0; j < cities_count; j++) {
            int fuel_to_city;
            std::cin >> fuel_to_city;
            fuel_between_cities[i][j] = fuel_to_city;
        }
    }

    int left = 0;
    int right = find_max(fuel_between_cities);

    while (left < right) {
        int mid = (left + right) / 2;
        if (can_fly_with_X_fuel(fuel_between_cities, mid, cities_count)) {
            right = mid;
        } else {
            left = mid + 1;
        }
    }

    std::cout << left << std::endl;
}

```

О решении:

Идея:

Условие возможности долететь из любого города в любой (возможно, с остановками) можно интерпретировать как проверка ориентированного графа на сильную связность. Эту проверку делаем с помощью обхода по ребрам и по обратным ребрам. Чтобы найти минимальный бак пользуемся бинарным поиском по ответу с проверкой на сильную связность.

Сложность по времени:

$O(n^2 \log C)$ – так как одна проверка для фиксированного X (топливо) стоит нам $O(n^2)$, потому что в обходе в ширину для каждой вершины просматривается вся строка или столбец, а бинарный поиск делает $\log C$ проверок

Сложность по памяти:

$O(n^2)$ – так как храним матрицу расхода топлива размером n на n